TRINITY COLLEGE · OCTOBER 26, 2026

AI Integration

Image generation, and the review

Ken Kousen · CPSC 415
Monday 1:30–4:10 · MC-225

Where we are

Last week

Your program read a photograph
The model read, your code checked
The agent wrote the tests; you proved they ran
A hook said no on your behalf

This week

Your program **makes** a picture
A model critiques it; your code checks the critique
A second model reviews the pull request
A gate stops a push that fails

> Stage 5 of the chain: Deploy. In the chain that means a reviewed merge, not hosting.

A second request shape

Chat (Weeks 2–5)

`POST /api/v1/chat/completions`
`messages` in, `choices[0].message` out
tokens priced in and out

Images (today)

`POST /api/v1/images`
`prompt`, `aspect_ratio`, `seed` in
`data[0].b64_json` + `media_type` out
`usage.cost` in the reply

- Embeddings had their own endpoint in Week 4. Images have their own too.
- `GET /api/v1/images/models` lists what the endpoint can call.

Some image models also answer on the chat endpoint with modalities=["image","text"]. Same model, two doors.

Raw, and through the SDK

```
url = "https://openrouter.ai/api/v1/images"
body = {"model": MODEL, "prompt": PROMPT,
        "aspect_ratio": "1:1", "seed": 415}
hdr = {"Authorization": f"Bearer {KEY}",
       "Content-Type": "application/json"}
req = urllib.request.Request(url,
      json.dumps(body).encode(), hdr)
d = json.load(urllib.request.urlopen(req))
b64 = d["data"][0]["b64_json"]
```

```
from openrouter import OpenRouter
client = OpenRouter(api_key=KEY)
r = client.images.generate(
  model=MODEL, prompt=PROMPT,
  aspect_ratio="1:1", seed=415)
b64 = r.data[0].b64_json
print(r.usage.cost)
```

- Same request, plumbing done, typed replies. A **dependency**: `pip install openrouter`, named in `spec.md`.
- It can hide an option: MiMo's `reasoning.enabled` has no SDK field; the SDK form is `reasoning={"effort": "none"}`.

Live A: five images, with the bill

```
python3 generate.py prompts.json --out images
```

```
images/poster-01.jpg  image/jpeg  seed=415  $0.014  3.3s
images/poster-02.jpg  image/jpeg  seed=416  $0.014  3.0s
images/poster-03.jpg  image/jpeg  seed=417  $0.014  2.8s
images/poster-04.jpg  image/jpeg  seed=418  $0.014  3.6s
images/poster-05.jpg  image/jpeg  seed=419  $0.014  4.5s
5 images, $0.0700 total, manifest in images/manifest.json
```

- `black-forest-labs/flux.2-klein-4b`: **$0.014** per 1:1 image from `usage.cost`, 3–4.5 s (measured October 20).
- The manifest records prompt, seed, model, media type, and cost **per image**, so nothing downstream guesses.

Seeds, and what "the same" means

- Same model + same prompt + same **seed** → the same picture, when the model honors the seed. Not always the same **bytes**: FLUX on Oct 20 gave the same picture twice and two different JPEG files.
- Different seed → a different picture of the same thing.
- That is what makes an image **set** reproducible and a billed test repeatable.
- The lab asks you to generate one prompt twice with one seed and say whether the picture matched, and whether the file did.

> A test that generates a fresh random image every run is not testing anything you can name.

Prompt and style control

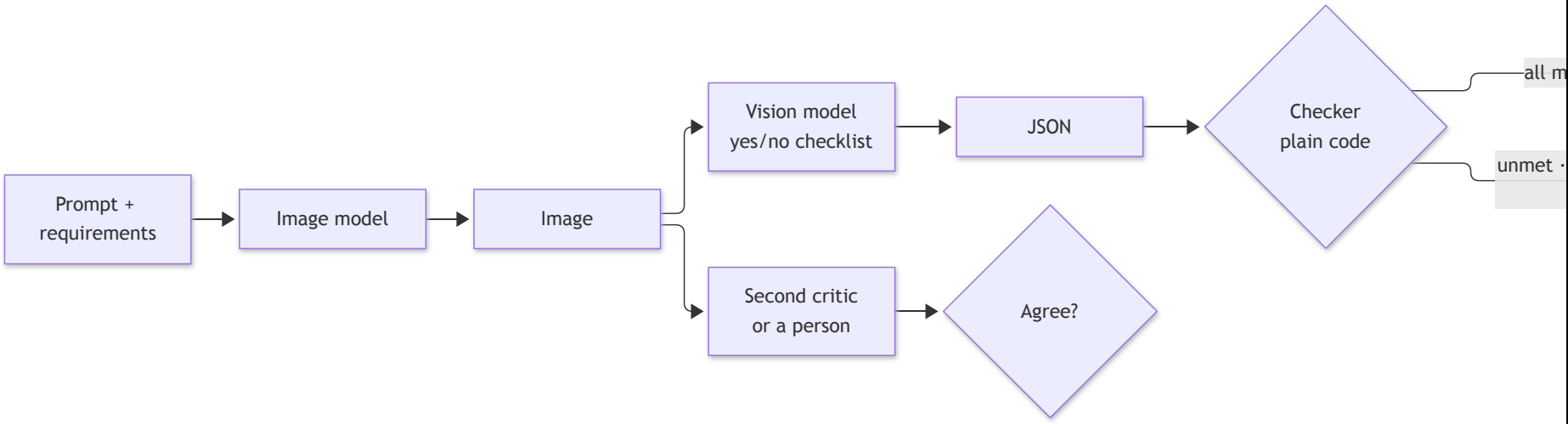
You say	The model does
Subject and scene: "a robot and a student at a whiteboard"	composes it
Style: "hand-drawn, marker on paper, warm colors"	renders in that style
Text: `the title "AI Integration", large, legible`	tries; text is where image models most often fail (3 of 5 posters on Oct 20)
`aspect_ratio`, resolution, output format	shapes the file
What to leave out: "no other text"	sometimes; a negative is weaker than a positive

Provenance: what a generated image carries

- Some generators embed an invisible watermark: `google/gemini-3.1-flash-lite-image` output carries **SynthID**, Google's watermark, detectable by their tools after cropping or recompression.
- FLUX output through OpenRouter carries no such mark; your **manifest** is its provenance: prompt, seed, model, date.
- Metadata in the file (EXIF, C2PA) can be stripped by any editor. The manifest you commit cannot be stripped by accident.
- Course rule, from the syllabus: no images of real people; generated media is labeled as generated where the API supports it, and in your README where it does not.

Provenance is a record of how the image came to be. Keep the record next to the image.

The loop



- The generator is a model. The critic is a model. **Only the checker is code.**
- The checker compares the critique with the requirements; it does not look at pixels.

The critique is a checklist, not an essay

```
Answer every item below with true or false, strictly from what is visible.
Reply with one JSON object and nothing else:
{"checks": [{"requirement": "<item, copied exactly>", "met": true, "evidence

- There is a robot in the image
- There is a whiteboard in the image
- The style looks hand-drawn, not photographic
```

The code then asks:

- Was **every** requirement answered? (missing)
- Was anything answered that was **not asked**? (invented)
- Was each one **met**?
- Was it JSON at all? (a non-JSON critique is a failed critique, not a crash)

Live B: the critic that was wrong

```
poster-02: ok  title read: 'AI INTEGATION' (MISMATCH)
poster-01: ok  title read: 'AI Integration' (match)
```

Open-ended *"Critique this poster"* on poster-01, MiniMax M3:

```
Oct 19: ... the title appears to be misspelled as "Integratlon" ...
Oct 20: # Critique of the "AI Integertion" Poster
```

- Poster-02: the generator misspelled it; the narrow read caught it. Poster-01: the generator was right; the critic invented a typo, a different one each day.
- The second critic read poster-02's `INTEGATION` as "AI INTEGRATION": it corrected what was printed.

> A critique is model output. Check it like any other.

Three ways to check the critic

Check	Catches	Costs
A **narrow question** compared in code: "write the title exactly" `==` expected	misread text, drifted answers	one small call per image
A **second model** on the same checklist; code lists disagreements	a critic that is confidently wrong alone	a second critique per image
A **person** looks at the disagreements	everything the models agreed wrongly on	your time, on the few images that disagree

- Agreement is not proof. Disagreement is a **pointer** to where to look.
- Team Project 1’s critique path should have at least the first.

Chain stage 5: Deploy

Stage	Artifact	Written by	Approved by
Plan	`intent/*.md`	agent, after interviewing you	you
Design	`spec.md`	agent, from the intent	you, against the intent
Build	`plan.md`, then code	agent	you, before any code
Test	tests, lint, the eval	agent	you confirm the loop ran
Deploy	**a pull request + review findings**	**agent; a separate agent reviews**	**the code owner merges**

Why a pull request, when an agent wrote the code

- A PR is the **unit of review**: one feature, one diff, one conversation, one decision.
- The agent that wrote the code has the same blind spots in review. A **different** reviewer is the point, model or person.
- The findings and the replies stay on the PR: that is the evidence for the chain.
- `main` is what the team runs. Nothing lands there without a second look.

```
git checkout -b week06-yourname
# ... tests first, then code ...
git push -u origin week06-yourname
gh pr create --base main --fill
```

REVIEW.md: four passes, ranked

Pass	Name	Examples
1	**Must not merge**	secrets in the diff; code that does not match `spec.md`; tests deleted, skipped, or weakened; destructive ops without a guard
2	**Must fix before merge**	model output used as fact without validation; missing handling for a failure the spec lists; no test for the behavior the PR claims
3	Should fix	duplicated logic, dead code, unclear names; unnecessary calls, a frontier model where the spec chose a cheap one
4	Nice to have	documentation and comments

How to report: one finding per line: severity, file and line, what is wrong, what to do instead.

The reviewer is a second model

```
python3 examples/pr-review/review.py; echo "exit $?"
```

```
Pass 1: Must not merge (2)
  generate.py:4: hardcoded API key committed -> read os.environ, rotate it
  generate.py:6-11: `make` never calls `critique`; spec requires it -> call
Pass 2: Must fix before merge (2)
  generate.py:10: json.loads(...)["verdict"] trusts model output -> validate
  generate.py:11: new `critique` behavior has no test -> add one
[minimax/minimax-m3 · 908 in / 1385 out · cost 0.0015]
exit 1
```

- In: `REVIEW.md`, `spec.md`, the diff. Out: findings by pass. Exit 1 on pass 1 or 2; exit 2 when it did not answer in form: a gate **fails closed**.
- It can be wrong both ways: check each finding against the code; a wrong one gets a reply on the PR.

Release gates: a rule the push cannot skip

Pre-push hook (your machine)

```
#!/bin/sh
set -e
ruff check .
env -u OPENROUTER_API_KEY \
  python3 -m pytest -q
```

`cp gate.sh .git/hooks/pre-push`

CI (GitHub Actions)

```
on: [pull_request]
steps:
  - run: pip install pytest ruff
  - run: ruff check .
  - run: python3 -m pytest -q
```

Required check on `main` → red cannot merge

- Free and fast on purpose: the key is removed, model tests skip. A gate that costs money gets switched off.
- Three hooks now: Claude Code's `PreToolUse` (the guard), git's `pre-push`, and CI. Different moments, same idea.

Team Project 1 on November 2

Submission	one member: repository URL + tag `tp1-submitted`, Moodle, 1:30 PM
Presentation	**10 minutes per team** + 5 of questions: the app live, one artifact on screen, one thing that broke
Order	drawn at the start of class
Peer sheet	each team gets written notes from the others; **format to be confirmed** by email before Nov 2
Chain	intent, spec, plan, tests, at least one reviewed PR **per member**, `ANNOTATION.md`

- This week's lab PR is one of those reviewed PRs.

Lab: generate, critique, check, review

0 Week 5 repo, `week06/`, SDK, rules

1 Intent and spec: generator, critic-checker

2 Plan, proof split, with the cost

3 Tests first, then the checker

4 Five images, five critiques, ~$0.15

5 **The critic was wrong:** second model, narrow question, your eyes

6 Branch, PR, reviewer, gate, merge, in the **team repo**

weeks/06/lab.md · about 60 minutes plus the PR review · due November 2 with TP1

Before November 2

- **Week 6 submission:** PR URL in the team repository + `week06-submitted` on your Week 5 repository.
- **Team Project 1:** complete, presented in class.
- **Assigned today:** the AI-Native SDLC Playbook course on Claude Academy; certificate due **November 9**. You have run every stage; now learn their names.
- Week 7 is presentations, then **audio**: speech in and speech out for the portfolio chatbot.

Before you leave

- What does the checker know that the critic doesn’t?
- Why does the reviewer fail closed when the guard fails open?
- What does the gate run that `REVIEW.md` can only read?

Sources and next steps

- OpenRouter: image generation · openrouter on PyPI · gh pr · Git hooks · Claude Code hooks.
- The AI-Native SDLC Playbook: the Deploy stage; the Academy course assigned today.
- Artifact-chain template: `REVIEW.md`, the four passes.
- Image generation, the critique, and the reviewer rehearsed October 19–20, 2026 on `black-forest-labs/flux.2-klein-4b`, `google/gemini-3.1-flash-lite-image`, `minimax/minimax-m3`, and `xiaomi/mimo-v2.6-flash`; prices quoted are those runs' `usage.cost`.

Prepared October 19–20, 2026.